CUSTOM OBJECTS

Scripting Programming Languages – Practical lesson 4

Julia Golubitskaya

TOPIC

Object is a collection of properties, each property consists of a string name and a value associated with that name.

```
let book = {
    "main title": "JavaScript",
    "sub-title": "The Definitive Guide",
    for: "all audiences"
};
```

TODAY

- Creating custom JavaScript objects
- Special operators: delete, new, in
- The for..in loop
- The hasOwnProperty() method
- 4 tasks

OBJECT BASICS

- **Object** is a collection of **properties**, each property consists of a string name and a value associated with that name.
- The value of a property can be a function, which is called an object's **method**.
- In addition to the built-in browser objects, you can define your own objects.
- Objects are used to store collections of various values and more complex entities.
- **Global Object** is a special JavaScript object, automatically created when the translator starts (before the execution of the first script). Properties and methods of the global object are accessible from anywhere in the script

OBJECT LITERAL

- The simplest way to create an object is by describing an object literal.
- **Object Literal** is a list enclosed in curly braces. Inside the curly braces, properties are specified separated by commas. Property is a "key: value" pair, where the key is a string (also called the "property name"), and the value can be anything.
- If an object literal is assigned as a value to a variable, then this variable is an object.

EXAMPLE: OBJECT LITERAL

```
let obj = {
    property: "this can be a string",
    // property name must be enclosed in quotes
    //if it consists of multiple words
    "second property": 15, // this can be a number
    method: function () {
        console.log("this can be a function");
    },
};
// accessing property via dot
document.write(obj.property);
// accessing property via square brackets
console.log(obj["second property"]);

obj.method();
```

OBJECT CONSTRUCTOR

Using literals, you can create objects in unlimited quantities, describing the literal for each object. But if there are many objects, and the set of properties and methods is the same for them, it makes sense to use an **object constructor function**. This is a function that will allow creating objects with the same set of properties. The constructor function essentially defines the type of object, its set of properties and methods.

To create a new instance of this object using the constructor function, it is necessary to call it with the **new** operator. The function call instruction is specified after the **new** operator. The action of the new operator is that an object is created, and then the constructor function is called, and a reference this to the created object is automatically passed to this function.

EXAMPLE: OBJECT CONSTRUCTOR

Thus, if we declare some variable and want to assign an object created using the constructor as its value, then the entire command might look like:

```
1  function Person(name, age) {       // create a constructor
2  function
3      this.name = name;              // assign values to
4  properties
5      this.age = age;
6      this.show = function () {
7          document.write(`Name: ${this.name} <br>`);
8          document.write(`Age: ${this.age}<br>`);
9      };
10 }
11 let obj1 = new Person("Yulia", 28); // create object instances
12 let obj2 = new Person("Polina", 9);
13 obj1.show();
14 obj2.show();
```

WORKING WITH OBJECTS

After creating an object, you can edit it.

```
let example = {}; // create an object without properties
// assign value to property
example.text = "Originality is our everything!";
example["print text"] = function () {
    console.log(example.text);
};
example["print text"]();
```

Using the **prototype** property, you can add new properties and methods to the object constructors.

```
Person.prototype.personality;
obj1.personality = "Super";
console.log(obj1.personality);
```

delete OPERATOR

The **delete** operator removes the given property from the object, returning true or false in case of success or failure.

- If the **delete** operator successfully performs deletion, it completely removes the property from the object. However, if a property with the same name exists in the prototype chain, the object inherits that property from the prototype.
- **Delete** is effective only when applied to object properties. It does not work with variable and function names. **Delete** cannot remove certain properties of built-in objects.

```
1  let student = { name: 'Polina', age: 18, class: 1 };
2  delete student.age;
3  console.log(student);
4  /* result: { name: 'Polina', class: 1 } */
```

in OPERATOR

- The **in** operator allows you to check whether a certain property or method is present in a particular object. If the property or method is contained in the object, then **true** is returned, otherwise - **false**. It follows that the **in** operator can be used in conditional expressions.

```
1  let obj = { name: "Ivan", age: 42 };
2  alert( "age" in obj ); // true, obj.age exists
3  alert( "date" in obj ); // false, obj.date does not exist
```

- Note that to the left of the in operator must be the name of the property. Usually this is a string in quotes. If you omit the quotes, it means that you are specifying a variable that contains the property name.

for..in LOOP

To iterate over all properties from an object, the for..in loop is used.

```
1  let basket = {
2      bananas: 5,
3      apples: 2,
4      title: "Fruits"
5  };
6  for (var key in basket) {
7      // this code will be called for each property
8      // of the object and will output the property name
9      // and its value
10     console.log("Key: " + key + " value: " + basket[key]);
11 }
```

hasOwnProperty METHOD

The **.hasOwnProperty()** method returns a boolean value indicating whether the object contains the specified property. Every object descended from Object inherits the *.hasOwnProperty* method.

This method can be used to determine whether an object contains the specified property as a direct property of that object; unlike the in operator, this method does not check for the existence of properties in the object's prototype chain.

EXAMPLE: hasOwnProperty METHOD

```
1  obj = new Object();
2  obj.prop = 'Property';
3
4  function changeO() {
5      obj.newprop = obj.prop;
6      delete obj.prop;
7  }
8
9  obj.hasOwnProperty('prop'); // will return true
10
11 changeO();
12 obj.hasOwnProperty('prop'); // will return false
```

TASK 1

Literal and Property Manipulation

1. Create an object ***smartphone*** using an object literal that contains:
- properties ***brand***, ***model***, ***price***;
- a method ***getDescription()*** that returns a string: "*Smartphone [brand] [model] costs [price] CNY*.".

2. After creation:
- call the **getDescription()** method and output the result to the console;
- add a new property ***color*** with a value, e.g., "black";
- change the value of the price property to a new one;
- delete the brand property using the delete operator;
- check with the in operator whether the property brand exists in the object and output the result;
- output the resulting object to the console.

TASK 1. EXAMPLE OF RESULTS

```
Description: Smartphone Apple iPhone 15 costs 5999 CNY.
After adding color property: {
  brand: 'Apple',
  model: 'iPhone 15',
  price: 5999,
  getDescription: [Function: getDescription],
  color: 'black'
}
After changing price to 4999: {
  brand: 'Apple',
  model: 'iPhone 15',
  price: 4999,
  getDescription: [Function: getDescription],
  color: 'black'
}
After deleting brand property: {
  model: 'iPhone 15',
  price: 4999,
  getDescription: [Function: getDescription],
  color: 'black'
}
Does brand property exist? false
Final object state: {
  model: 'iPhone 15',
  price: 4999,
  getDescription: [Function: getDescription],
  color: 'black'
}
```

TASK 2

Constructor Function

1. Create a constructor function ***Movie*** that takes parameters:
- title (movie title);
- director (director);
- year (year of release).

2. In the constructor:
- assign the passed values to the object’s properties;
- add a method ***getInfo()*** that returns a string:
- "*Movie: [title], Director: [director], Year: [year]*".

3. Create two instances:
- "The Shawshank Redemption" (dir. Frank Darabont, 1994);
- "Inception" (dir. Christopher Nolan, 2010).

4. Call the ***getInfo()*** method for each movie and output the result.

TASK 2. EXAMPLE OF RESULTS

```
First movie created: Movie {
  title: 'The Shawshank Redemption',
  director: 'Frank Darabont',
  year: 1994,
  getInfo: [Function (anonymous)]
}

Second movie created: Movie {
  title: 'Inception',
  director: 'Christopher Nolan',
  year: 2010,
  getInfo: [Function (anonymous)]
}

Movie 1 info: Movie: The Shawshank Redemption, Director: Frank Darabont, Year: 1994
Movie 2 info: Movie: Inception, Director: Christopher Nolan, Year: 2010
```

TASK 3

Prototypal Inheritance

Based on the Movie constructor from the previous task:

1. Add a method ***setRating(rating)*** to the Movie prototype.

2. This method should set a rating (a number from 1 to 10) for the movie. The property rating should be added directly to the object:

- for the first created movie, set a rating of 9.5 using the new method;
- for the second movie, set a rating of 8.7.

3. Output information about each movie together with its rating in the console in the format:
"*Movie: [title], Director: [director], Year: [year], Rating: [rating]*".

Hint: add a new method ***getFullInfo()*** *that calls* ***getInfo()*** *and appends the rating.*

TASK 3. EXAMPLE OF RESULTS

```
First movie created: Movie {
  title: 'The Shawshank Redemption',
  director: 'Frank Darabont',
  year: 1994,
  getInfo: [Function (anonymous)]
}

Second movie created: Movie {
  title: 'Inception',
  director: 'Christopher Nolan',
  year: 2010,
  getInfo: [Function (anonymous)]
}

After setting rating 9.5 for first movie: Movie {
  title: 'The Shawshank Redemption',
  director: 'Frank Darabont',
  year: 1994,
  getInfo: [Function (anonymous)],
  rating: 9.5
}

After setting rating 8.7 for second movie: Movie {
  title: 'Inception',
  director: 'Christopher Nolan',
  year: 2010,
  getInfo: [Function (anonymous)],
  rating: 8.7
}

First movie full info: Movie: The Shawshank Redemption, Director: Frank Darabont, Year: 1994, Rating: 9.5

Second movie full info: Movie: Inception, Director: Christopher Nolan, Year: 2010, Rating: 8.7
```

TASK 4

Iterating over Properties

1. Create an object laptop:

```
let laptop = {
    brand: "Dell",
    model: "XPS 15",
    ram: 16,
    ssd: 512
};
```

2. Perform the following steps:

- Use a ***for..in loop*** to output all own properties of the object to the console in the format: "*Property: [key], Value: [value]*".
- Add a property category with the value "Electronics" to the Object prototype.
- Run the ***for..in loop*** again and output all properties (including inherited ones).
- Modify the loop so that it outputs only own properties (using ***hasOwnProperty***) and output the result again.

TASK 4. EXAMPLE OF RESULTS

```
Loop 1 - Own properties only (before adding to prototype):
Property: brand, Value: Dell
Property: model, Value: XPS 15
Property: ram, Value: 16
Property: ssd, Value: 512

Added category property to Object prototype

Loop 2 - All properties including inherited:
Property: brand, Value: Dell
Property: model, Value: XPS 15
Property: ram, Value: 16
Property: ssd, Value: 512
Property: category, Value: Electronics

Loop 3 - Own properties only using hasOwnProperty:
Property: brand, Value: Dell
Property: model, Value: XPS 15
Property: ram, Value: 16
Property: ssd, Value: 512

Final object state: { brand: 'Dell', model: 'XPS 15', ram: 16, ssd: 512 }
Check if category is own property: false
Check if category exists in object: true
```

TEST QUESTIONS

1. What is an object in JavaScript?
2. What is the difference between creating an object using an object literal and using a constructor function?
3. What does the delete operator do? Give an example of when it might fail.
4. What is the purpose of the hasOwnProperty() method?
5. How do you iterate over all enumerable properties of an object, including inherited ones?
6. What does the new operator do when creating an object with a constructor function?
7. How can you add a new method to all existing and future instances of a constructor function at once?

THANK YOU FOR ATTENTION

Scripting Programming Languages – Practical lesson 4

Julia Golubitskaya